TSIS: Optimization in Deep Learning

OBJECTIVES

Deep Learning - Optimization Algorithms Exercise

- Practice implementing a small Multi-Layer Perceptron (MLP) from scratch or using a framework.
- Understand the practical differences between various optimization strategies.
- Explore one of two experimental tracks:
  - Compare convergence speeds of different algorithms (SGD vs. Adam).
  - Analyze the effect of Batch Size (Stochastic vs. Mini-batch vs. Full Batch).

PREREQUISITES

Basic Python and deep learning at an introductory level. You should understand what a Loss Function is and the basic concept of Backpropagation.

BRIEF THEORY (VERY SHORT)

Training a neural network is essentially an optimization problem: finding the set of weights that minimizes a Loss Function. You can imagine this as a hiker trying to walk down a mountain into a valley (the minimum loss) while blindfolded, feeling only the slope of the ground under their feet.

- **Gradient Descent:** The hiker takes a step in the steepest direction.
- **SGD (Stochastic):** The hiker takes a step based on a rough guess of the slope (using one data point).
- **Momentum/Adam:** The hiker runs down the hill, gathering speed and momentum to roll over small bumps.

LIBRARIES AND DATASET USED

Environment

- Python 3 with PyTorch, TensorFlow/Keras, or NumPy (if coding from scratch).
- CPU is enough.

Dataset

- You will use a simple synthetic dataset or a lightweight classification dataset (e.g., MNIST or Fashion-MNIST).

INDEPENDENT EXERCISE (3-4 HOURS)

Step 1: Choose your project track

Pick one of the two options:

Option A: Algorithm Showdown (SGD vs. Adam vs. RMSprop)

- Fix your batch size (e.g., 32) and model architecture. You will run the training loop three times, changing only the optimizer.
- **Goal:** See which algorithm converges (reaches low loss) fastest.

Option B: The Batch Size Effect (Stochastic vs. Mini vs. Full)

- Fix your optimizer (e.g., Standard SGD) and learning rate. You will run the training loop three times with different batch sizes:
  - Batch Size = 1 (Stochastic)
  - Batch Size = 32 or 64 (Mini-batch)
  - Batch Size = Size of Dataset (Full Batch Gradient Descent)

Step 2: Design a simple pipeline

In your notebook or script, clearly separate:

1. **Data Generation / Loading**
   - If using Synthetic Data: Generate a noisy sine wave or a 2D spiral classification problem.
   - If using MNIST: Load the data, normalize it, and flatten images to 1D vectors (784 inputs).
2. **Model Definition (The MLP)**
   - Define a small MLP. It should be simple: Input Layer, Hidden Layer (e.g., 32 or 64 neurons) with ReLU activation, and Output Layer.
3. **The Training Loop function**
   - Write a function `train_model(optimizer_name, batch_size, learning_rate)` that trains the model for a fixed number of epochs (e.g., 10 or 20) and returns a list of **Loss values** recorded at every step or epoch.
   - *Crucial:* Ensure you re-initialize the model weights before every new run so that every optimizer starts from the same starting point.

Step 3: Run and document experiments

1. Run your chosen Option (A or B).
2. Collect the Loss History for each run.
3. Plot the results on a single graph:
   - X-axis: Epochs (or Iterations).
   - Y-axis: Training Loss.

- You should have 3 lines of different colors (e.g., Blue for SGD, Red for Adam, Green for Momentum).

DELIVERABLES

1. **Code notebook or script that runs end-to-end:**
   - Data setup.
   - Model class.
   - The comparison loop.
   - The final visualization plot.
2. **Short report (1-2 pages) with:**
   - A few sentences describing your chosen Option (A or B).
   - The plot of your Loss Curves.
   - A short analysis: Which line went down the fastest? Did any line ”jitter” or bounce around?

CONTROL QUESTIONS

You should be able to answer these in simple language:

1. What is the difference between ”Stochastic” Gradient Descent and ”Full Batch” Gradient Descent?
2. Looking at your plot, which optimizer or batch size provided the smoothest curve? Which was the noisiest?
3. Why does the Adam optimizer usually converge faster than standard SGD?
4. If you set the Learning Rate too high, what happens to the Loss curve? (Did you observe this?)
5. Which parts of the code did you get help from GPT/LLMs for?
6. Did the fastest optimizer also give the highest final accuracy, or just the fastest drop in loss?
7. What is one limitation of using a batch size of 1 (Stochastic)?
8. If you had more time, how would you tune the Learning Rate for each optimizer differently?